



— HTML CAPSTONE

Build a Programme Page

Bringing structure, media, and forms together in a complete, accessible document.

The brief: one useful page

Your task is to build a complete, accessible HTML page for a new LEA Labs programme. No CSS, no JavaScript—just pure, semantic structure that communicates meaning to browsers and assistive technologies.

1 Semantic Navigation

A header with an unordered list of descriptive links to sections within the page.

2 Content Hierarchy

A logical flow of headings (h1, h2, h3) introducing the programme and its outcomes.

3 Data Presentation

An accessible table displaying the course schedule and module breakdown.

4 Interactive Form

A robust registration form with explicit labels, native validation, and clear button intent.

TARGET PAGE STRUCTURE

`<header>` Navigation

`<main>` Programme Intro & Media

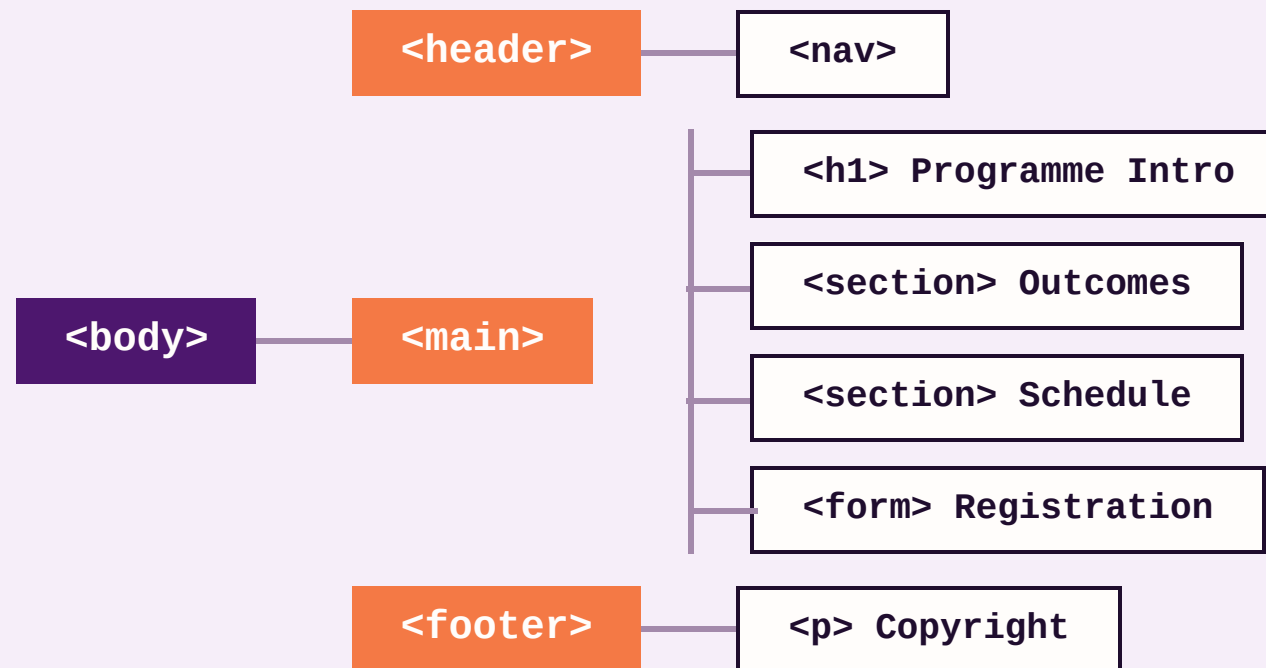
`<section>` Outcomes & `<table>` Schedule

`<form>` Registration

`<footer>` Legal & Contact

Map the page before coding

Never start typing tags blindly. Map out the semantic landmarks and document flow first. This guarantees your hierarchy makes sense before you get lost in the details.



Step 1: Start with a valid document

Every HTML page requires a standard foundation. This shell tells the browser how to interpret the document, sets the language for screen readers, and separates metadata from visible content.

`<!DOCTYPE html>`

Forces the browser to render the page in standards mode, preventing unpredictable legacy behaviors.

`<html lang="en">`

The root element. The `lang` attribute is critical for screen readers to pronounce words correctly and for translation tools.

`<head>`

Contains metadata. The `charset` ensures characters display correctly, and the `viewport` tag makes the page responsive on mobile devices.

`<body>`

The container for all visible content. Everything the user sees or interacts with goes inside this element.

INDEX.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width, initial-scale=1.0">
  <title>Software Engineering | LEA Labs</title>
</head>
<body>
  <!-- Visible content goes here -->
</body>
</html>
```

Step 2: Create the site header

The `<header>` landmark introduces the page, and the `<nav>` element contains the primary navigation. We structure links as an unordered list because a menu is semantically a list of choices.

`<header>`

The introductory landmark. It typically contains the site logo and the main navigation menu.

`<nav>`

Defines a major block of navigation links. Using `aria-label` helps distinguish it if there are multiple navs.

`<ul>` & `<li>`

Groups the links logically. This tells screen readers exactly how many navigation items are present.

`<a href="#id">`

Anchor tags with fragment identifiers (#) allow users to jump directly to specific sections on the same page.

INDEX.HTML (INSIDE `<BODY>`)

```
<header>
  <a href="/"></a>
  <nav aria-label="Primary">
    <ul>
      <li><a href="#about">About</a></li>
      <li><a href="#outcomes">Outcomes</a></li>
      <li><a href="#schedule">Schedule</a></li>
      <li><a href="#register">Register</a></li>
    </ul>
  </nav>
</header>
```

Introduce the programme

The `<main>` landmark holds the core content. It begins with a single `<h1>` to establish the page's primary topic, followed by a summary and a clear call to action.

The Main Landmark

Every page should have exactly one `<main>` element. It wraps the unique content of the document, excluding headers, footers, and sidebars.

The Primary Heading

Use exactly one `<h1>` per page. It tells search engines and screen readers exactly what the page is about.

The Call to Action

Use an `<a>` tag to direct users to the registration form. Even if it looks like a button later with CSS, semantically it is a link.

PROGRAMME INTRO HTML

```
<main>
  <article>
    <h1>Software Engineering</h1>

    <p> A practical route into software engineering for
    learners who want to understand how digital products are
    planned, built, and tested. </p>

    <a href="#registration">Enroll Now</a>
  </article>
</main>
```

Explain outcomes and requirements

Break down the programme details using semantic `<section>` elements, `<h2>` headings, and unordered lists to make the content scannable and accessible.

SEMANTIC SECTIONS

Use `<section>` to group related thematic content. Give each section an `id` so the navigation links can jump directly to them.

HEADING HIERARCHY

Since these sections are direct children of the `<main>` (which contains the `<h1>`), their titles must be `<h2>` to maintain a logical outline.

SCANNABLE LISTS

Never use paragraphs with line breaks (`<br>`) for lists. Use `<ul>` and `<li>` so screen readers can announce the number of items.

PROGRAMME DETAILS HTML

```
<section id="outcomes">
  <h2>What You Will Learn</h2>
  <ul>
    <li>Build semantic HTML structures</li>
    <li>Create accessible web forms</li>
  </ul>
</section>

<section id="requirements">
  <h2>Prerequisites</h2>
  <p>No prior coding experience required. You need:</p>
  <ul>
    <li>A modern web browser</li>
    <li>Curiosity and persistence</li>
  </ul>
</section>
```

Add a real schedule table

Tables are for tabular data, not layout. We will build an accessible schedule using proper table semantics so screen readers can announce row and column headers correctly.

`<caption>`

The title of the table. Essential for accessibility, it must be the first child of the table.

`<thead>` & `<tbody>`

Semantic wrappers that group header rows separately from the primary data rows.

`<th scope="col">`

Explicitly tells assistive tech that this header applies to the entire column below it.

`<th scope="row">`

Defines a header for a specific row, ensuring data cells are read with their row context.

INDEX.HTML (SCHEDULE SECTION)

```
<section id="schedule">
  <h2>Programme Schedule</h2>
  <table>
    <caption>16-Week Module Breakdown</caption>
    <thead>
      <tr>
        <th scope="col">Week</th>
        <th scope="col">Module</th>
        <th scope="col">Focus</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <th scope="row">1-4</th>
        <td>HTML & CSS</td>
        <td>Structure & Style</td>
      </tr>
    </tbody>
  </table>
</section>
```

Give media a purpose

Images should add meaning, not just decoration. We use the `<figure>` element to group an image with its caption, and provide concise `alt` text for screen readers.

`<figure>`

A self-contained container for content like images, diagrams, or code snippets that are referenced in the main flow.

The `alt` Attribute

Mandatory for all images. Describe the *meaning* of the image. If the image is purely decorative, use an empty string (`alt=""`).

`<figcaption>`

Provides a visible caption for the figure. Screen readers automatically associate this caption with the image it describes.

INDEX.HTML (MEDIA SECTION)

```
<section id="gallery">
  <h2>Campus Life</h2>
  <figure>
    <!-- Always include width and height -->
    
    <figcaption>
Students building their first HTML
capstone project.
    </figcaption>
  </figure>
</section>
```

Build the registration form

Forms are how users talk back to your site. A semantic form ensures every input is clearly labeled, grouped logically, and validated natively by the browser.

Explicit Labels

The `for` attribute on a `<label>` must exactly match the `id` on its corresponding `<input>`.

`<fieldset>` & `<legend>`

Groups related form controls together. The legend provides a caption for the entire group.

Input Types

Using `type="email"` triggers the correct mobile keyboard and provides basic native validation.

Native Validation

The `required` attribute prevents submission if the field is empty, without needing JavaScript.

INDEX.HTML (REGISTRATION SECTION)

```
<section id="registration">
  <h2>Register Interest</h2>
  <form action="/submit" method="POST">
    <fieldset>
      <legend>Your Details</legend>

      <label for="name">Full Name</label>
      <input type="text" id="name" name="name" required>

      <label for="email">Email Address</label>
      <input type="email" id="email" name="email" required>
    </fieldset>

    <button type="submit">Submit Registration</button>
  </form>
</section>
```

Assemble the complete page

We have built every piece of the programme page. Now, we combine them into a single, valid HTML document. Before adding CSS, always verify your semantic structure.

1. DOCUMENT SHELL

Valid DOCTYPE, `<html lang="en">`, and proper metadata in the `<head>`.

2. LANDMARK FLOW

Content is logically divided into `<header>`, `<main>`, and `<footer>`.

3. HEADING HIERARCHY

Exactly one `<h1>` per page, with `<h2>` and `<h3>` nested sequentially without skipping levels.

4. ACCESSIBLE COMPONENTS

Images have `alt` text, form inputs have associated `<label>`s, and tables use `<th scope>`.

INDEX.HTML (STRUCTURAL VIEW)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Meta & Title -->
</head>
<body>
  <header>
    <!-- Logo & <nav> -->
  </header>

  <main>
    <article>
      <h1>Software Engineering</h1>
    </article>
    <section id="outcomes">...</section>
    <section id="schedule">...</section>
    <section id="media">...</section>
    <section id="register">...</section>
  </main>

  <footer>
    <!-- Copyright & Links -->
  </footer>
</body>
</html>
```

Capstone lab: publish meaning first

Build a complete, semantic HTML-only landing page for a new LEA Labs programme. Focus entirely on structure, accessibility, and meaning.

YOUR MISSION

Create an HTML-only page for a fictional "Data Science" programme. Do not use any CSS or JavaScript. Rely purely on HTML tags to convey the document's structure.

CORE DELIVERABLES

- A valid HTML5 document shell with correct metadata.
- Semantic landmarks (<header>, <main>, <footer>).
- An accessible schedule using a <table> with proper scopes.
- A registration <form> with native validation and linked labels.

QUALITY CHECKS

- Passes the W3C HTML Validator with zero errors or warnings.
- All form inputs have explicitly associated <label> elements.
- All images have meaningful alt text (or empty if decorative).
- Heading hierarchy strictly follows logical order (H1 → H2 → H3).

EXTENSION CHALLENGE

Group related form fields (e.g., "Personal Information" vs "Course Preferences") using <fieldset> and <legend> elements to provide even richer context for assistive technologies.